

Air traffic control as an MDP

Session #2 — Advanced, part 1: the problem, the simulator, and what is wrong with it

Tutorial track

RL Bootcamp 2026 · Salzburg, 16–18 September

Flight4DEnv board measured at VERSION 0.29.0; data/air_4d_board.csv

Why this problem

Ninety minutes, in four parts

🕒 10 min

-
- 10 Why air traffic control, and why a toy version of it
 - 30 The simulator, and the MDP you inherit: observation, actions, how an episode ends, the reward, the score
 - 20 Do you actually need RL? A hand-written controller, introduced pedantically, and the board it sets
 - 15 What is wrong with the current solution, and where you could take it
-
- 15 Questions. Then the second deck: how to compete.
-

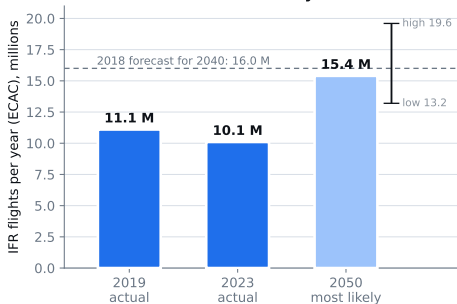
⚠️ Everything here is written down

These slides are pictures and pointers. Every derivation and every number lives in the handbook, open in a tab while you work:

```
docs/advanced/airtraffic.md
docs/reference/environment-api.md
docs/reference/reward-design.md
docs/reference/rule-based-controller.md
```

Europe's sky is filling up. The number of controllers is not.

+39% on 2019 by 2050



EUROCONTROL Aviation Long-Term Outlook 2024-2050 (Dec 2024);
Challenges of Growth 2018. Actuals derived from the published growth rates.

⚠ The bottleneck is a person

- Airspace is cut into **sectors**, and each sector is one controller's attention
- Its capacity is declared in aircraft per hour — a *human workload limit, not a runway limit!*
- More traffic means smaller sectors, more controllers, or **more aircraft per head.**

What the controller's job is, per aircraft

Watch it. Predict who it will meet. Issue a **clearance** — a level, a speed, a heading — by voice over radio

Watch it again. Every clearance is a decision whose consequence arrives minutes later, among twenty other aircraft

They're all remarkable people

An assistant, not a replacement

💡 Why this is an RL-shaped problem

- Sequential decisions
- Late consequences
- Trade-off *per aircraft* between safety, punctuality and how much you intervene
- A simulator that can replay a million afternoons. The agent **proposes**; the controller **disposes**

Prototype stage, with the operators in the room

- Low TRL SESAR 3 JU exploratory research
- **Validated** with working controllers — workshops, interviews, human-in-the-loop sims
- The industrial partners are already in the consortia (an ANSP, an ATM system vendor)
- *Path from theory to an ops room*

Where the field actually is

TADA — Terminal Airspace Digital Assistant

- An agent proposes an **optimised 4D trajectory per arrival**
- TMA entry to final approach, aligned with the arrival manager
- Explainable HMI — the key to **increasing trust** with RL-based assistants in ATM
- Target: **lower controller cognitive workload** at higher capacity
- *University of Malta, Ingenav, Deep Blue, Monad.*

ASTRA — tactical hotspot prediction & resolution

- Predict en-route traffic complexity hotspots **60 minutes** ahead, then resolve them
- RL agents with a multi-headed policy issue speed, flight-level and direct-routing clearances
- Performed **validation exercise with real controllers** — they say it's a promising start
- *University of Malta, Ingenav, Deep Blue, Skysoft-ATM, Skyguide.*

Abstracts and links: sesar.ju.eu → SESAR Solutions → TADA! · sesar.eu → News → AI-enabled solution ASTRA.

Why I wrote airtraffic: the real problem, with almost everything removed

	In the ops room	In this package
Routes	waypoints, arcs, procedures	straight lines , entry to exit
Headings	the controller's most-used tool	never commanded — every track is fixed
Pilots	readback, negotiation, seconds of delay	obsequious : the clearance lands this step
Fuel	a climb costs; descend-and-climb costs twice	free . Altitude has no price at all
Separation	5 NM laterally, 1000 ft vertically	1000 m and two levels of 12
Clock	minutes to an encounter, hours on shift	100 s episodes, 2 s steps
Weather, noise	always	none. Deterministic from a seed

⚠ The one thing kept in place of fuel

- With altitude free, “park everyone on a different level” is a perfect policy
- **secondary objective: timeliness**
- Every aircraft entered with a predetermined *4D exit point*
- A clearance is a detour, not a new destination.
Leave it deviated and you pay

💡 What survives the simplification

- The *shape* of the decision: who to touch, with what, and when
- Consequences a dozen steps later
- A hard safety rule that outranks everything else. Enough to make the design questions real, and small enough to train in an hour.

Straight lines. Obedient pilots. Free altitude.

Everything that is left is the real problem.

The simulator, and the MDP you inherit

How a scenario is born from a seed

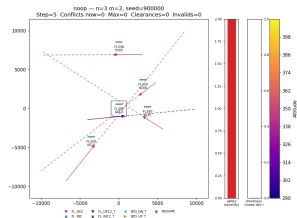
30 min

CollisionCourseSimulator(n , m)

1. A hotspot in the central ± 1 km box; a shared level.
2. n aircraft placed *on* it — random heading, one of ten speeds — then **rewound** out of the box.
3. m background aircraft at *other* levels, rewind the same way.
4. The clock shifted forward *uniformly* until everyone is on screen.

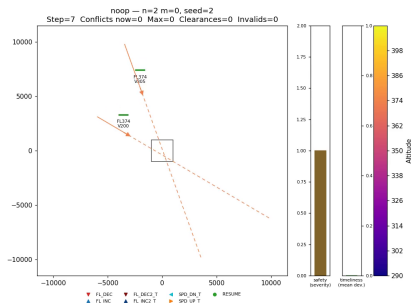
💡 $n + m = 5$ always; $n \sim U\{2, 3, 4\}$ each reset

The sector size fixes the spaces; only the *split* varies.
`reset(seed=k)` gives everyone the same traffic based on seed k



do-nothing — $n=3$, $m=2$, seed 900000: three converge on the square, two cross elsewhere [▶ play](#)

How to read a render



do-nothing, two aircraft and nothing else — seed 2, mid-air at

step 18 [play](#)

solid track

dashed line

dashed circle

solid circle

black square

coloured tick

two bars

markers

where it has flown, coloured by the level held

where it *will* go if untouched — a fact, not a guess, and exactly as long as it has left to fly

a predicted encounter, still legally separated

a loss of separation *now*; thicker with severity, fading after

the ± 1 km box the convergence point is drawn in — *not* the conflict radius, which is pairwise

that aircraft's 4D deviation, green $\rightarrow$ red

live safety and timeliness cost, the reward's own numbers

a clearance, stamped where it was issued: $\blacktriangle$ $\blacktriangledown$ level, $\blacktriangleright$ $\blacktriangleleft$ speed, $\bullet$ resume

The title carries the running count of conflicts, clearances and invalid actions — the cost of a resolution is on screen while you watch it.

Loss of separation is counted. A mid-air ends the episode.

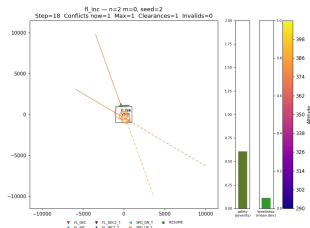
⚠ In conflict, counted every step, iff *both*

$|\Delta_{alt}| < 24$ **and** horizontal separation ≤ 1000 m.

A procedural violation. You are charged for every step it persists, and you fly out of it. This is the safety KPI.

💀 A mid-air, and the episode is over

Two aircraft within **100 m** and less than one level apart. The only genuine terminal state: nothing after it can undo it, so its value is zero and the meter stops.



one level apart, 7 m apart: five conflict-steps, and everybody lives



💡 Read the two together

Two aircraft one level apart at 7 m are *not* a mid-air — and they are in conflict for every step of the encounter. That gap between the two rules is where most of the design lives.

The action: one aircraft, then, one clearance

	FL_DEC	FL_INC	FL_DEC2_T	FL_INC2_T	SPD_DN_T	SPD_UP_T	RESUME
aircraft 0	1	2	3	4	5	6	7
aircraft 1	8	9	10	11	12	13	14
aircraft 2	15	16	17	18	19	20	21
aircraft 3	22	23	24	25	26	27	28
aircraft 4	29	30	31	32	33	34	35
idle	0						

action = 1 + 7×aircraft + clearance | Discrete(36)

FL_DEC / FL_INC

one level, **permanent**

FL_DEC2_T / FL_INC2_T

two levels, hold 5 steps, **revert**

SPD_DN_T / SPD_UP_T

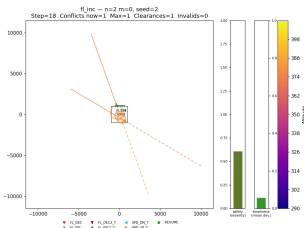
∓ 35 m/s for 5, ± 35 for 5, then nominal — a **symmetric pulse**

RESUME

entry level *and* entry speed back, in one instruction

- At most **one aircraft per step**
- Every **_T** variant means it's active for **T** steps only — the recovery is part of the instruction and it is charged **once**.
- The environment takes one number; the baseline *policy* chooses it in two stages (idle, or which aircraft; then which clearance — explained in a few slides)

Same pair, same step, one instruction — one level, or two

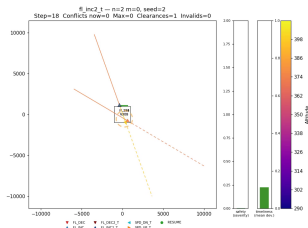


FL_INC at step 14 — 5 conflict-steps, **no mid-air**, exit_miss

0.500 [play](#)

⚠ One level is not a fix

The pair still passes 7 m apart horizontally — one level keeps it out of the mid-air rule and *inside* the conflict rule, for five steps. And the aircraft never comes home: half the sector is a level off plan at the exit.

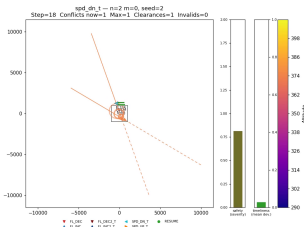


FL_INC2_T at step 14 — 2 conflict-steps, **no mid-air**,

exit_miss 0.000 [play](#)

💡 Two levels delete the conflict, and the return is booked

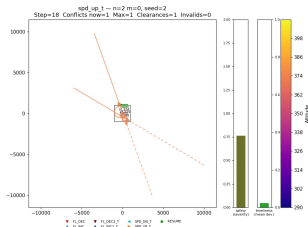
ALT_MIN_SEP reached: the pair leaves the conflict list. Watch hold_left count down and the aircraft descend by itself — one clearance, charged once, and exit_miss zero.



SPD_DN_T at step 14 — passes at 258 m, 5 conflict-steps,
exit_miss 0.000 [▶ play](#)

! 70 m per step

Four steps of -35 m/s move the crossing by 280 m : a wide miss in mid-air terms, still squarely inside the 1000 m radius. Speed is the slow tool — it needs *time*, which a late clearance does not have.

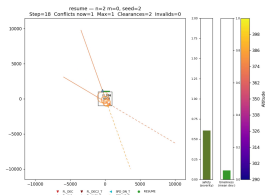


SPD_UP_T at step 14 — passes at 274 m, 4 conflict-steps,
exit miss 0.000 [▶ play](#)

💡 Schedule-neutral by construction

– dv for five steps, + dv for five, then nominal: the along-track debt is repaid inside the same clearance and the 4D exit is untouched. Nobody has to remember to speed back up.

RESUME closes the loop — and the eight clips, in numbers



FL_INC at 14, RESUME at 24 — 5 conflict-steps, 2 clearances, exit_miss 0.000 [▶ play](#)

clip	ends	conflicts	clear.	exit_miss
do-nothing	18	3	0	0.000
FL_INC / FL_DEC	50	5	1	0.500
FL_INC2_T / FL_DEC2_T	50	2	1	0.000
SPD_DN_T / SPD_UP_T	50	5 / 4	1	0.000
FL_INC + RESUME	50	5	2	0.000

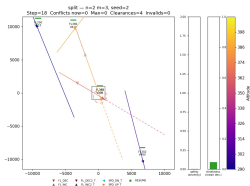
⚠ Same seed, same aircraft, same step. Only the instruction differs.

Do-nothing is perfectly timely and dead. One level is safe by the letter and late by half a level. **Every column can be won by a different clip** — and the scoring, next, ranks them rather than adds them.

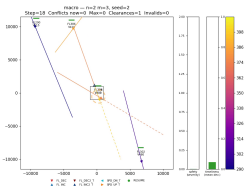
```
render_4d.py --n-flights 2 --n-range 2 2  
--seed 2
```

— pinned by `tests/test_render_4d.py`.

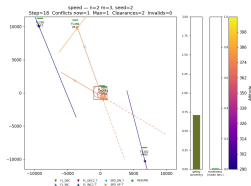
The same conflict with bystanders — three resolutions that trade differently



split the levels — 0 conflict-steps, 4
clearances [▶ play](#)



one macro — 2 conflict-steps, 1 clearance
[▶ play](#)



offset the speeds — 5 conflict-steps, 2
clearances, `exit_miss 0.035` [▶ play](#)

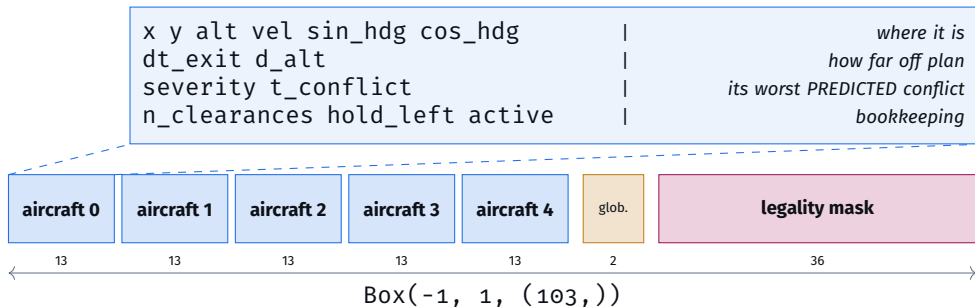
⚠ Seed 2 again, now with three background flights ($n=2, m=3$)

Split: FL_INC one, FL_DEC the other, then a RESUME each once past — what the rule-based controller does, and the it's the simplest approach in this simulator

Macro: a quarter of the price, and it does not fully clear the encounter.

Speed: nobody changes level; one arrives early and one late. Timing each of these is most of what separates a good agent from a bad one.

The observation: one flat vector, and the mask rides inside it



⚠ One frame is enough, and severity/t_conflict are PREDICTED

Headings are never commanded, so every track is a straight line and a single frame is **Markov** — no stacking, no recurrence. The conflict features are not “are we in conflict now” but *how bad, and how far away*: a conflict twenty steps out is already in the vector. The two globals are the clock and the budget — both end the episode, so both are observable.

The mask: illegal moves are removed, not punished

Three families of illegality — `action_masks()`

Rails. A climb at the ceiling, a descent at the floor, $\pm dv$ at the speed limits. The macro needs *two* levels of headroom

Inertness. RESUME on an aircraft already on plan; any clearance to one mid-manoeuve. Budget spent on nothing.

Kinematic dead ends. A speed change after which *no* legal speed still reaches the 4D exit slot. Nothing later recovers it, so it is gone.

⚠ Why it lives *inside* the observation

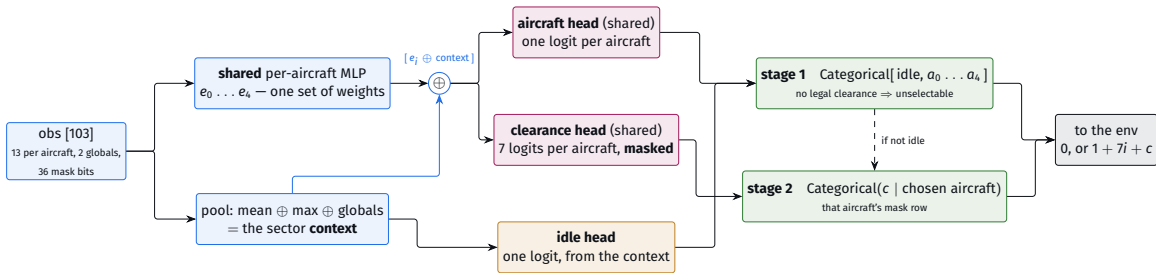
- An action mask is present when the action is chosen and **absent when its log-probability is recomputed**
- PPO then optimises a ratio between two different distributions, as a wrong answer rather than an error
- `obs[mask_offset:] > 0.5` is the mask; the policy and the controller both read it there

💡 Invalid actions are made inert, not punished

An **action issued against the mask is a no-op** that increments `info["invalids"]` — counted so a broken policy is loud

We do not charge invalid actions — an invalid move is not something we have to learn since the simulator already knows it

The baseline policy: idle, or which aircraft — then which clearance



💡 Two consequential choices, one flat index

Do nothing, or pick an aircraft — then **what to tell it**, masked for *that* aircraft. `idle_bias = 2.5` puts $p(\text{idle}) \approx 0.71$ at initialisation, or a fresh policy spends the budget before the conflicts arrive. The mode is the *joint* argmax, not greedy stage by stage.

⚠️ What the factorisation does and does not buy

With a shared stage-two MLP, i then $c \mid i$ is **distributionally identical** to one flat 36-way categorical over $\log p(i) + \log p(c \mid i)$ — a test asserts it. The win is **masking conditioned on the chosen aircraft**, and a place to hang a conditional later.

Terminated is not truncated, and the difference is a bootstrap

terminated — a mid-air

Two co-level aircraft within 100 m. The episode is **over**: there is no future to value, so the value target is the terminal charge and nothing else.

truncated — horizon, or budget

50 steps, or 40 clearances spent. The aircraft are still flying; the *episode* stopped, not the world. SB3 bootstraps $V(s_T)$ here, and it must.

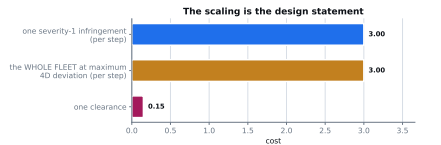
⚠ Get this backwards and the agent learns to crash

Every reward term is a **cost**, so ending early **stops the meter**. If the terminal charge is smaller than the cost of flying on, dying is the optimal policy.

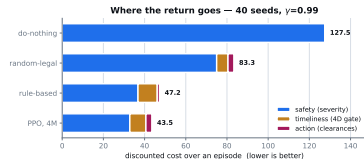
The charge is therefore $\text{collision_rate} \times w_{\text{safe}}$ per *remaining* step — **450** at step 0, against a worst cost-of-continuing of **170.55**.

`calibrate_4d.py` checks that margin on every run. Re-run it after any weight change.

The reward: three direct costs, and nothing else



Safety and timeliness are deliberately EQUAL: one pair in full conflict outweighs the whole sector being as late as it can be. One clearance is $w_{\text{safe}}/20$.



do-nothing pays zero timeliness and zero action cost — and still loses by $3\times$.

💡 The scaling IS the design statement

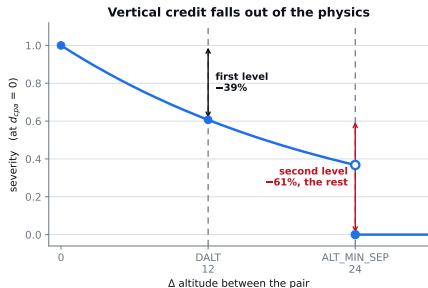
One severity-1 infringement per step = the whole fleet at maximum 4D deviation = 20 clearances. A single pair in full conflict outweighs every aircraft being as late as it can be.

⚠️ Deliberately NOT potential-based

Predicted severity is already dense, so shaping buys nothing. The price: a direct cost **can** move the optimum, where shaping provably cannot. Retuning w_{safe} changes the *answer*, not just the search.

In the lower chart: do-nothing pays **zero** timeliness and **zero** action cost, and still loses by $3\times$.

Where the safety credit comes from — and why one level pays 39%



The drop at ALT_MIN_SEP is a deletion, not a decay — the pair leaves the conflict list. One clearance moves an aircraft by DALT, so a single climb buys 39% and resolves nothing.

⚠ A deletion, not a decay

At $\Delta h = H$ the pair leaves the sum entirely. One level buys $1 - e^{-1/2} = 39\%$ and resolves **nothing**; the second buys the remaining **61%**. A *linear* credit, $1 - \Delta h/H$, paid 50% for the first — and the policy bought exactly that, on every aircraft, at step 1.

💡 The credit is not chosen — it falls out of the conflict rule

For every pair $i < j$ that *will* come within R before the horizon, with $\Delta h_{ij} < H$:

$$\sigma_{ij} = \exp\left(-\frac{d_{ij}^{cpa}}{R}\right) \exp\left(-\frac{\Delta h_{ij}}{H}\right)$$

$$\text{severity}(s) = \sum_{i < j} \sigma_{ij}, \quad r_{\text{safety}} = -w_{\text{safe}} \text{severity}(s)$$

Pairs with $\Delta h_{ij} \geq H$, or whose tracks never close to within R , contribute exactly **0**. Charged every step, so a conflict twenty steps out is already in today's sum.

$R = D_SEP = 1000$ m, the loss-of-separation radius · $H = ALT_MIN_SEP = 24$, two flight levels ·

$\Delta h_{ij} = |\text{alt}_i - \text{alt}_j| \cdot d_{ij}^{cpa}$ = horizontal distance at the pair's **closest point of approach** on

their current straight tracks: with relative position $\mathbf{r}_0$ and relative velocity $\Delta \mathbf{v}$,

$t_{cpa} = -\mathbf{r}_0 \cdot \Delta \mathbf{v} / |\Delta \mathbf{v}|^2$ clipped to the window inside R and to the steps left in the episode, and

$d^{cpa} = |\mathbf{r}_0 + \Delta \mathbf{v} t_{cpa}|$ (predict_conflicts).

What is scored — and it is not the reward

(failed_episodes, congestion, timeliness_bucket, clearances, touched)
_____ safety _____ timeliness _____ efficiency _____

Lexicographic, not weighted

Compare failures first. Only ties go on to congestion; only ties on both go on to the 4D gate; and so on.

A safer submission outranks a cheaper one **by any margin whatsoever**. Timeliness is bucketed to 0.005 so that ties genuinely occur and efficiency is not decoration.

⚠ Why failed comes before congestion

An episode that ends in a mid-air stops accumulating conflict-steps **purely by existing for less time**. Rank on safety alone and crashing out becomes a strategy. Failing to fly the sector is not a way to fly it safely.

`exit_miss = mean(|late| / one hold +
|levels off|)` — an untouched aircraft is at zero by construction.

The reward is yours. The ranking is not.

We score you on our KPIs, on our seeds, never on your reward.

Do you actually need RL?

Before you train anything: the two policies that need no training

20 min

do-nothing — the brick

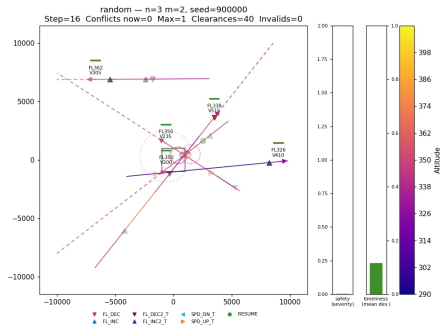
Never issues a clearance, so it is **perfectly timely and perfectly efficient by construction**. It kills everyone with 95% confidence

random-legal — the action space's own score

Uniform over whatever the mask allows. **What your action space scores for free**; nothing below it has learned anything.

⚠ Why both, every time

The brick bounds what inaction buys; random bounds the *action space*. An agent can lose to each, on a different column (KPI).



random-legal — $n=3, m=2$, seed 900000
40 clearances buy 4 conflict-steps and exit_miss 1.035
Result: a clean-ish sector with every schedule in it wrecked

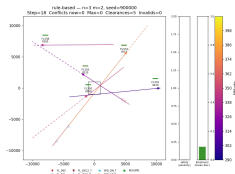
play

A controller you can write in an afternoon: assign, steer, come home

Priority4DController knows nothing privileged

The same flat vector the network sees, inverted to physical units by `decode()`; the mask read off `obs[mask_offset:]`.

1. **Assign slots, once, at reset.** Converging traffic is whoever has a predicted conflict. Earliest arrival keeps its level; later arrivals get base ± 2 , ± 4 from a list built *once*, so no two share a slot.
2. **Steer, one level per step.** Most urgent first (soonest `t_conflict`), FL_INC/FL_DEC toward its slot; if masked, the next aircraft.
3. **Come home when genuinely clear.** At 7 hotspot radii from *every other* converging flight, one RESUME — then retire the slot, or it flies straight back out.



rule-based, same seed — 5 clearances, 0 conflict-steps,
`exit_miss` 0.400 [▶ play](#)

⚠ The bug worth knowing about

base + offset, *clipped* into the band,
collapses +2 and +4 onto one level near a
rail: two aircraft told to sit in the same slot.
8 of 10 remaining failures.

Why the controller refuses the macro — the most transferable result here

It looks strictly dominant

Two levels — the minimum that resolves anything — for one clearance instead of three, and the return is free.

20 failures in 40 episodes — why

displaced, reverted on the timer, **hit**
never commanded at all (budget spent elsewhere)
hit *while still displaced*

A macro-only controller: **50%** failures against random-legal's 34%.

Sweeping the lead time 1–6 steps never got below 50%.

⚠ The return time is fixed when you issue it

The revert fires inside `step()`, before the next observation. Converging traffic is *sustained* proximity and outlasts `hold_steps`: “come back on a timer” means “come back into the traffic”. Zero failures while displaced — the return leg is the whole problem.

💡 The general lesson

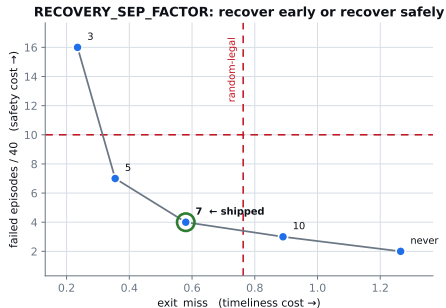
A macro bundles *go* and *come home* into one price: a bargain when the right return time is knowable at issue time, a trap when it is not. A learner that can *predict* it may use the macro well. A fixed timer cannot.

12

8

0

The controller's one knob is a safety-against-timeliness trade



Down-left is better on both. 7.0 is the last setting that beats random-legal on every column.

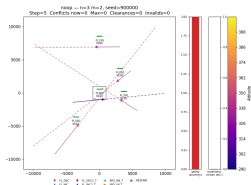
RECOVERY_SEP_FACTOR, swept over 40 seeds

Both axes are costs. Recover early and exit_miss improves while failures rise; recover late and the sector is safe with aircraft parked off-level. **7.0** is the last setting that beats random-legal on every column.

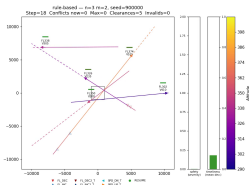
💡 The trade a learned policy has to win

One global radius for every aircraft in every geometry, no re-planning, and the speed axis never touched. Per-aircraft decisions from the visible geometry are the headroom.

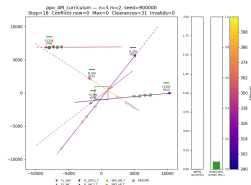
Same scenario, three agents — watch before you read a number



do-nothing — collides, step 12 [▶ play](#)



rule-based — clean, 5 clearances [▶ play](#)



PPO, 4M — clean, 31 clearances [▶ play](#)

⚠ Same $n=3$ $m=2$ sector, same seed, three different afternoons

The table on the next slide says the same thing in numbers — **watch it once first**. do-nothing dies before it has a chance to be efficient; the other two never let a pair close. What they spend to get there is the whole design problem.

The board — 100 held-out seeds, and every row wins a column

>_ Run it

```
cd sessions/03-advanced/airtraffic
python scripts/score_4d.py --agent noop --agent random --agent rule-based # ~2 min
```

#	agent	failed	ep_len	cong.	/100 steps	conflict-free	exit_miss	clear.
1	PPO masked, 4M [det]	1	49.1	170	3.46	49%	0.087	2278
2	PPO masked, 4M [sto]	2	49.1	225	4.58	48%	0.143	3338
3	rule-based — the bar	14	44.8	90	2.01	63%	0.554	667
4	random-legal	34	35.1	528	15.05	9%	0.754	2924
5	noop — the brick	96	12.5	551	44.04	4%	0.000	0

⚠ The learner wins the ranking

One failed episode against fourteen, and exit_miss $6.4\times$ lower — the first environment in this bootcamp where a learner beats the hand-written bar.

⚠ And loses the column it looks like it won

170 against 90 on comparable episode lengths: the controller is $1.7\times$ cleaner per step flown, for under a third of the clearances. **Claim what you measured, on which column.**

So — do you need it? What a good answer sounds like

💡 Evidence, not allegiance

“PPO underperforms the heuristic on per-step congestion because it spreads clearances over twice as many aircraft. It wins the 4D gate because it can decide the return time per aircraft, which one global radius cannot.”

⚠️ The failure to avoid

*“Deep RL always wins” is an assumption. So is “a heuristic is always enough.” Both are cheap to test here, and the test is the same one: **score them on the same held-out seeds and read every column.***

When the heuristic wins

Predictable structure, few interacting agents, a rule you can write down. Deterministic, no training, and you can debug it.

When RL earns its place

Many simultaneous conflicts, edge cases nobody enumerated, and a trade-off surface — *per aircraft* — that no single global constant can express.

[docs/advanced/baseline.md](#) ·

[docs/reference/rule-based-controller.md](#)

**What is wrong with it, and where
you could take it**

Where the learner still loses — read the whole row

🕒 15 min

	PPO 4M	rule-based
failed episodes, of 100	1	14
conflict-steps / 100 steps flown	3.46	2.01
conflict-free episodes	49%	63%
exit_miss	0.087	0.554
clearances, 100 episodes	2278	667
aircraft touched	482	205

100 seeds do not pin a failure rate. The same checkpoint fails **21 of 600** further held-out seeds ($\approx 3.5\%$). The row is reproducible; the *rate* needs more seeds than a board has.

⚠️ **It buys safety with clearances it should not need**

3.4× the instructions, 2.4× the aircraft touched, 1.5× as many per aircraft touched — for a sector that is *dirtier* per step. The controller assigns a slot and finishes; the agent keeps re-touching.

💡 **Which is the third objective, unsolved**

Safety and timeliness are won; efficiency is not. The ranking reaches it last, so it only separates agents tied on the first two — which is exactly where the top of the board will be decided.

The commitment problem — measured on the first design, open on this one

injected	helps	nothing	hurts
1 altitude clearance	0.9%	96.9%	2.2%
2 altitude clearances	17.3%	81.2%	1.6%
1 MACRO clearance	17.3%	81.2%	1.6%

One clearance injected into an all-idle rollout; 46,080 injections, 24 seeds — **on the retired first-generation environment**, whose macro was permanent. The macro reproduces the two-clearance row exactly, at half the price.

💡 The probe was right

One level is not a weak move but the *wrong* one: mean effect +0.053, the wrong sign. The macro made a resolving move reachable in one decision instead of a p^2 coincidence.

⚠️ And the training run said no — same environment

Two 1.5M-step runs, identical except the action set. On 50 frozen seeds the **macro run ranked below the control**: 6 failures to 2, and **45 clearances per episode against 8**. Cheap in *decisions* did not make the policy selective; it made it prolific.

What that leaves open — on Flight4DEnv, for you

Here the macro is *temporary* and the probe has not been re-run. What the board does say: the trained agent issues **4.7** clearances per touched aircraft against the controller's **3.3**, on $2.4\times$ as many aircraft. Over-insurance, or indecision? Unmeasured. Action repeat, a longer hold_steps, a “finish what you started” feature, a commitment cost — all untried. ^{32/63}

The lies in the model — and whether fixing each one travels to the scored run

Simplification	What it hides	What you could do	Scored?
no heading clearances	the controller's main tool	a vector action — and the geometry stops being a straight line	no
altitude is free	fuel; climb and descent are not symmetric	price the vertical axis — the reward is yours	yes
pilots comply this step	readback delay, refusals, partial compliance	a delay or a compliance probability, in training	training
fixed revert timers	the return time is decided at issue time	a variable-hold macro, or a learned RESUME	yes
one aircraft per step	a controller issues several a minute	multi-aircraft commands, and the exploration cost they carry	yes
deterministic, no wind	every geometry exactly predictable	wind, drift, sensor noise — domain randomisation, in training	training
100 s, five aircraft	sustained workload; traffic arriving mid-episode	a longer horizon with entries and exits — a bigger design job	yes

⚠ The physics are frozen; the interface is yours

CollisionCourseSimulator is identical for everyone, or the ranking means nothing. Observation, action, reward, termination, network and training regime are open. A change to the dynamics is legal **during training** and does not travel to the scored run.

Six tracks — pick one, change one thing, predict one number

Track	Ideas
A Observation	relative (Δx , Δy , Δalt) to the conflict partner · time to closest point of approach · slack against the 4D gate · graph representation
B Action	split the macros back into single levels and measure what it costs · a longer or variable <code>hold_steps</code> · a permanent speed clearance · more than one aircraft per step
C Reward	potential-based shaping beside the direct costs · annealed clearance charge · a commitment cost · Lagrangian constraint · counterfactual credit for a resolving clearance
D Dynamics	stochastic headings · wind field · observation noise · compliance delay — <i>training only</i>
E Masking	ablate it. Which of the legality rules is actually carrying the result?
F Architecture	the autoregressive policy in <code>policies/</code> — beat it, or ablate it

💡 Write it down before you train: “I would change X, and I expect Y to happen to the KPIs.”

The prediction is the important half — it makes the afternoon an experiment instead of a rummage. A change that made things worse, understood, beats one that helped for reasons you cannot name.

`docs/advanced/design-sprint.md`

Eleven mistakes we made building this — every one catchable with arithmetic

- **1–3** the reward said something we did not mean: a bonus that pays everyone; a stock charged as a rate; a reward that scored **0.39%** of what it penalised
- **4–5** the optimum was right, the path was not: 99.3% of first moves out of idle hurt; one level never resolves
- **6–7** the action space was not what we thought: 86:1 in favour of acting; a mask refusing 84% of speed clearances
- **8–9** the measurement was wrong: 20 episodes inverted a ranking; a deterministic rollout of an idle-biased policy *is* the brick
- **10–11** shaping and the scored rule disagreed: a linear credit for a threshold rule; a crash cheaper than flying on

💡 The habit underneath all eleven

Before a training run, write down what each term is worth, when it is paid, and how often it fires. Compare those numbers to each other. Every mistake here is a comparison nobody made.

⚠️ Each has its own frame in the appendix

Extra — eleven mistakes: the arithmetic that caught it, what it did to the policy, and the line it put in `Flight4DEnv`. Jump there during questions or the sprint.

Before you touch anything — five habits that cost minutes and save afternoons

>_ Run it

```
cd sessions/03-advanced/airtraffic
pytest                                     # the contract, <30 s
python scripts/calibrate_4d.py           # what each reward term is worth, <1 min
python scripts/score_4d.py --agent noop --agent random --agent rule-based # the bar, ~2 min
```

💡 While it trains

1. Read `ep_len_mean` first — if it is not heading to 50, every other number is deflated by dying.
2. Score the **frozen seeds**, never the training callback; 20 episodes have inverted a ranking here.
3. Score `[det]` and `[sto]` — the gap between them is the entropy.

⚠ Before it trains

4. Size the reward terms against each other. A weight wrong by 86:1 is knowable in thirty seconds.
5. Put a random legal policy in every table. **An agent worse than random on an objective is telling you that objective is not priced.**

The commands, the flags, the submission and the board:
next deck.

The sector is yours.

Questions — then the second deck: how to compete.

Extra — the problem in more depth

The constants, and the two relations that decide what is possible

Constant	Value	Meaning
DT	2 s	one step; 50 steps = 100 s
AIRSPACE	± 10 km	the box
D_SEP	1000 m	separation radius
COLLISION	100 m	a mid-air
levels	10, 290–398	DALT = 12 apart
ALT_MIN_SEP	24	two levels
speeds	10, 200–515 m/s	DVEL = 35 apart
hold_steps	5	a temporary manoeuvre
max_clearances	40	the budget

 **DALT = 12 against ALT_MIN_SEP = 24**

One altitude clearance can never separate a pair.

It moves them $0 \rightarrow 12$, still inside the threshold.

The minimum effective vertical intervention is **two levels**.

 **DVEL $\times$ DT = 70 m per step**

Against a 1000 m radius a speed change needs **≈ 14 steps** to open separation on its own. Speed is *slow*, and a 50-step episode has room for about three such manoeuvres end to end.

Ten levels at two-level separation give exactly **five** conflict-free slots — and at most four aircraft converge. Solvable, and only just.

The 4D exit gate — a clearance is a detour, not a destination

Because heading is fixed, the exit *point* cannot move. Only the **time** of arrival and the **level** can. So the gate reduces to a distance to run and the time that distance takes at the entry speed:

$$t_i^* = \frac{s_i}{v_i(0)}, \quad \Delta t_i = \hat{t}_i - t_i^*$$

where $\hat{t}_i$ is the arrival time projected from the current speed.

An aircraft you never touch has $\Delta t_i = 0$ **by construction**. Untouched traffic is free.

⚠ What this closes

There is a trivially safe policy: **slow everything down**. Decelerating de-synchronises the arrivals and the conflicts simply stop happening. It would win a safety-only leaderboard and it is operationally worthless.

💡 And what it deliberately allows

Detour freely. What costs you is detouring and **never coming back**. An aircraft already at $v_{\max}$ cannot make up lost time, so *which* aircraft you delay is part of the problem.

The tension at the centre of the problem

A speed clearance resolves conflicts by shifting an aircraft's whole *temporal* trajectory. Hold $+\Delta v$ for H steps:

$$\delta s = \Delta v H \Delta t_{\text{step}}, \quad \delta t = \delta s / v_0$$

Returning to the nominal **speed** does not return the **schedule**. δt is permanent, and recovering it needs an exact, equal-and-opposite clearance later.

⚠ Why that is hard for a learner

The payoff for the first half arrives only if the second follows. Sampling independently each step, a matched pair is a p^2 coincidence — and the first half is punished meanwhile.

💡 The design question this poses — and how we answered it

One instruction or two? FLIGHT4DENV made SPD_UP_T a **symmetric pulse** — $+\Delta v$ then $-\Delta v$, as *one* clearance. Zero net schedule shift, and the pairing never has to be discovered. The counter is $2H + 1$, not $2H$: the tick happens inside the issuing step, and $2H$ leaves one $\Delta v \cdot \Delta t$ of debt nothing repays. **Disagree? That is Track B.**

Case study: the first design — and what Flight4DEnv replaced

Observation	$11 \times k_frames \times n_flights$ $+ 4 \times a_frames + 3$ 593 values at the scored $n=5, m=5$
Action	MultiDiscrete([5] * n_flights) NOOP · VEL_DEC · VEL_INC · FL_DEC · FL_INC
Reward	$-10 \times \frac{conflicts}{n-1} - \frac{clearances}{100}$ $- \frac{invalids}{50} - \log(1 + \text{exit dev.})$
Ends	50 steps · 100 clearances · 50 invalids mid-air — the only true terminal state

Three blocks, all in $[-1, 1]$

Flights: pose, velocity, heading, time-to-exit, **predicted**-conflict severity and timing, signed exit-gate error. **Actions:** your last 10 clearances. **Globals:** clock, budget, invalid headroom.

Note what is already a design choice

Every aircraft commanded every step, 5^n joint actions, and a near-uniform policy bankrupt by **step 13**. Moving to Discrete($1 + 7n$) — one aircraft per step — is the single change that took this environment from untrainable to trainable. The interface, not the policy, was the binding constraint.

The most expensive mistake of the first design

Measured on the design that was replaced

Conflicts were not spread evenly: a scenario carried **40.3** loss-of-separation events, and **62.7%** of them landed at step 15 or later. Episodes ended at step **13**.

FLight4DEnv removes the cliff by construction — one clearance per step, so the budget cannot be burned in a dozen steps.

⚠ Put the two halves together

An untrained policy issues ≈ 8 clearances per step and the environment truncates at 100. So it sees **6.3 of 40.3** conflicts and reaches the horizon in **0%** of episodes.

It is not failing to solve the problem; **it is not being shown it.**

💡 On any episodic task, before you tune a reward

Plot *where* your outcome occurs and *where* your episodes end. If the second is left of the first, **stop tuning and fix that.**

Critique it — 10 minutes, in pairs

Questions to argue about

- 103 numbers for 5 aircraft. Is any of it redundant — and is anything *missing* that you would want as a controller?
- Positions are absolute; a conflict is a *relative* geometry. Whose frame should the observation be in?
- One aircraft per step. Is that the right interface?
- A mid-air ends the episode; running out of clearances only truncates it. Is that distinction doing what you want?

💡 Write down one change and one prediction

“I would change X, and I expect Y to happen to the KPIs.”

The prediction is the important half — it makes the afternoon an experiment instead of a rummage.

⚠️ Keep them

We come back to these at the post-mortem. Predictions written down before you measured are worth ten made afterwards.

Extra — what you get to design

What is a design decision — and six ways to get it wrong

The four you always make

What does it observe? Full state or partial?

How much history? In whose frame?

What can it do? How granular? Reversible? One actuator per step, or all of them?

What does it optimise? Dense, sparse, shaped — with what weights?

How long is an episode? γ , the horizon, what ends it early.

These four are not independent. Change one thing per experiment — you will want to change three.

Pitfall	What you see
Reward hacking	high reward, useless behaviour
Observation aliasing	cannot separate two states
Action mismatch	slow learning, or uncontrollable
Sparse reward	nothing to climb
Misleading shaping	confident, and wrong
Mis-specified γ	unstable or myopic

Every one is in this session, measured

Reward hacking is mistake 1. Sparse-that-never-fires is mistake 4. Misleading shaping is the step-function potential. Action mismatch is mistake 7.

Five methods, and that is the whole interface

```
class MyEnv(gym.Env):
    def __init__(self):
        self.observation_space = spaces.Box(
            -1.0, 1.0, shape=(4,), dtype=np.float32)
        self.action_space = spaces.Discrete(3)

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)  # seeds np_random
        return self._get_obs(), {}

    def step(self, action):
        return obs, reward, terminated, truncated,
        info
```

The reward is one number the algorithm consumes. **info** is what a *human* needs to judge the policy — it is where your KPIs live.

terminated vs truncated

terminated — the MDP ended: goal reached, or the state is absorbing. Bootstrapping stops here.

truncated — an outside limit cut it short. The value function *should* still bootstrap.

Get this backwards and your critic learns that running out of time is as bad as crashing.

Wrappers, and the contract that gates a submission

>_ Before you change anything, and after

```
cd sessions/03-advanced/airtraffic && pytest
```

```
# the contract, <30 s
```

What the suite pins down

Spaces have the declared shapes; `reset(seed=k)` reproduces the same scenario every time; the same seed and clearances reproduce the reference trajectories **exactly**; every KPI key the scorer reads is in `info`; termination fires where it should.

⚠ This is the gate, not a suggestion

A submission whose environment fails the contract suite is **not scored** — and it is also why you *can* redesign freely.

The space choice that bites

The first design used `MultiDiscrete([5] * n)` — every aircraft commanded every step, 5^n joint actions. `Flight4DEnv` uses `Discrete(1 + 7n)`: **at most one aircraft touched per step**, 36 actions at $n=5$.

Same problem, wildly different exploration. It is the single change that moved this environment from untrainable to trainable — the interface, not the policy, was the binding constraint.

The network is a design decision too — and it ships as the default

⚠ What is wrong with a flat `MlpPolicy` here

Traffic is a **set**, not a vector. A flat head gives flight 0's clearance its own weights, separate from flight 4's — “an aircraft converging on the hotspot should descend” has to be learned once *per slot*. Swap two aircraft and the flat policy sees an unrelated observation for a physically identical situation.

`policies.AutoregressivePolicy`

```
obs                                [B, 13·5+2+36]
→ reshape to flights              [B, F, 13]
→ shared encoder                  [B, F, E]
→ mean+max pool, globals [B, 2E+2]
→ stage 1: IDLE, or WHICH aircraft
    (1+F way; no legal move: masked)
→ stage 2: WHICH clearance for it
    (C way, that aircraft's mask row)
```

Reorder the flights and the outputs reorder identically. Permutation equivariance holds **by construction**, not by training. Illegal moves are masked at the logits — invalid actions go to **exactly zero**.

MaskablePPO cannot express every mask you will want

sb3-contrib's MaskablePPO consumes a **flat** boolean array over the whole action space. That covers *“may this aircraft be commanded?”*.

It cannot express a mask that depends on a **sampled part** of the action — *“which clearances are legal, **given** the aircraft just chosen”*.

Once the action factorises and the second factor's validity depends on the first, the flat interface breaks down: mask **inside a custom policy** on stock PPO. That is what Flight4DEnv does — the mask rides in the observation.

If you do want MaskablePPO (Track E)

```
from sb3_contrib import MaskablePPO
from sb3_contrib.common.wrappers \
    import ActionMasker

def mask_fn(env):
    return env.unwrapped.action_masks()

env = ActionMasker(env, mask_fn)
model = MaskablePPO("MlpPolicy", env)
```

⚠ Design questions it raises

What is “in conflict” — $\text{time-to-CPA} < T$, $\text{separation} < D$, both? And does masking help more on easy or hard scenarios?

**Extra — eleven mistakes, and the
design they produced**

Eleven mistakes we made — and FLight4DEnv is the fix list

Every one of these was written by someone who had thought about it, shipped into a training run, and found later. Most cost hours of compute. All of them were **catchable with arithmetic before launching**.

They are grouped by what the mistake was *about*, because the classes recur and the specifics will not:

- **1–3** the reward said something you did not mean
- **4–5** the optimum was right; the path to it was not
- **6–7** the action space was not what you thought
- **8–9** the measurement was wrong
- **10–11** the shaping and the scored rule disagreed

💡 The habit underneath all eleven

Before a training run, write down what each term is worth, when it is paid, and how often it fires. Compare those numbers to each other.

Every mistake here is a comparison nobody made.

⚠️ Not a war-story reel

FLight4DEnv is what you get when you fix all eleven. Each frame ends with the line it put in the code.

1-3 — the reward said something we did not mean

Mistake	The arithmetic that catches it	What it did
1. a bonus that pays everyone	A +100 survival bonus paid to anything reaching the horizon, do-nothing included. At $n=2$ $m=3$ it was +100.00 of the brick's +75.04 total .	The largest term in the return was the one no policy could influence . Removing it: brick -524.00 , controller -13.89 .
2. rate, or stock?	$n_clearances/100$ evaluated <i>every step</i> . One issued at step 5 is charged 45 more times; one at step 45, five. A factor of nine for the same command .	It taught the agent to defer intervention — the exact opposite of air traffic control. Nothing is wrong locally; the defect exists only in the episode sum.
3. the reward disagreed with the score	Reward charges the deviation every step; the KPI reads it <i>once</i> , at the end. Over 100 seeds: 9.963 summed against 0.039 scored - 0.39% .	Reward says <i>never leave your plan</i> ; the ranking says <i>leave it freely, but come back</i> . Every policy learned not to deviate rather than to recover .

💡 → what Flight4DEnv does

No survival bonus and no terminal bonus at all — three costs and nothing else. The clearance charge is paid **once, on the transition that issues it**. And `dt_exit` is in the observation, so the quantity the third objective scores is the quantity the agent can see and act on.

The problem was never that the optimum was in the wrong place.

99.3% of first moves out of doing nothing make things worse.

4-5 — the optimum was right; the path to it was not

injected	helps	nothing	hurts
1 altitude clearance	0.9%	96.9%	2.2%
2 altitude clearances	17.3%	81.2%	1.6%
1 MACRO clearance	17.3%	81.2%	1.6%

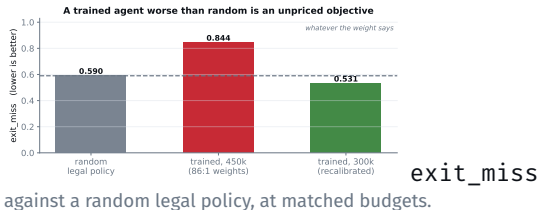
One clearance injected into an all-idle rollout; 30,720 injections, 24 seeds.

→ FL_DEC2_T/FL_INC2_T book **two levels in one instruction**: the two-clearance row, for one clearance. And the reward was never the bottleneck — $P(\text{reward} \leq 0 \mid \text{congestion improved}) = 0.0\%$ in every row. **No reward can fix a threshold; only the action space can.**

4. Every wrong guess is punished immediately; the payoff needs the right aircraft, clearance *and* step. So it never starts — **an exploration failure in a reward failure's clothes.**

5. DALT=12 vs ALT_MIN_SEP=24: one level leaves the pair *in conflict*. Not weak — **wrong.**

6 — a weight is not a strength, and the check that costs one rollout



⚠ Two asymmetries a weight cannot see

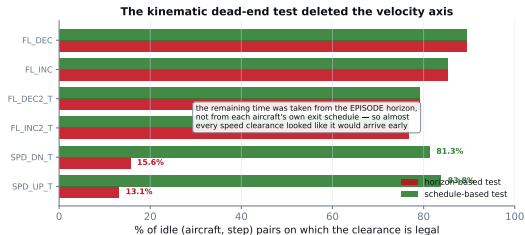
Per-event vs per-episode. A conflict is worth w_{safe} *once*; clearances are paid *each*. One-to-one gave **86:1 in favour of acting** — 26 clearances an episode, and **exit_miss 0.844** against random's 0.590.

Immediate vs terminal. ϕ_{safe} pays now; ϕ_{4D} telescopes to the gate and arrives multiplied by $\gamma^T \approx 0.75$.

💡 Score a random legal policy first

An agent worse than random on an objective is telling you that objective is not priced. One rollout loop; we ran it third.

7 — a mask that refuses everything is a missing action



⚠ One wrong quantity

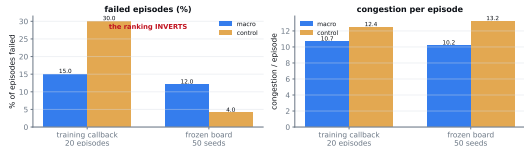
The dead-end test asked “*can this aircraft still make its gate?*” but took the remaining time from `max_steps` — the **episode** horizon — not the aircraft’s own t_i^* . Most leave long before step 50, so nearly every speed clearance looked like it would arrive early. **13–16%** legal; fixed, **81–84%**.

💡 → the mask lives *inside* the observation

`Box(103, )` at $n=5$ is $13n$ features + 2 globals + **36 legality bits**. SB3’s rollout buffer stores observations and nothing else — a mask passed any other way is present when the action is chosen and **absent when its log-probability is recomputed**, so PPO optimises a ratio between two different distributions.

8-9 — the measurement was wrong

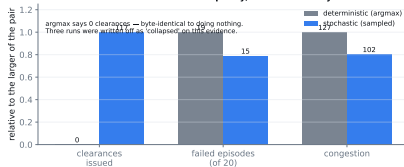
The same two agents, scored two ways



8.

On the 20-episode training callback the macro agent led on failures *and* congestion. On 50 frozen seeds **the ranking inverts**. We reported the callback result live, three times.

The same untrained policy, scored two ways



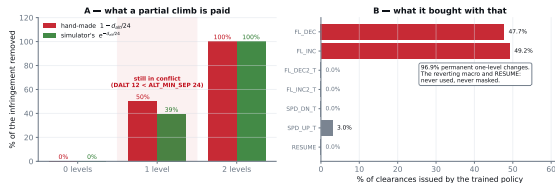
9. The policy

starts biased towards idling, so **idling is the argmax** — a deterministic rollout of an untrained policy is byte-identical to doing nothing. **Three runs were written off as “collapsed to NOOP” on that evidence. All three were learning.**

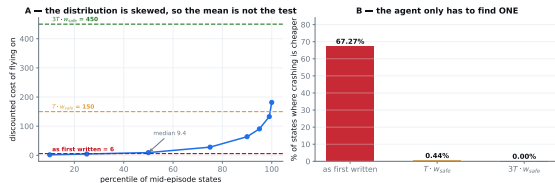
💡 → **score_4d.py** scores 100 held-out seeds, **det** and **sto**

Seeds 900000–900099, a block held away from the training callback's. The gap between the two modes is the entropy: when it closes the policy has settled; when it is wide, the argmax is not the policy.

10–11 — the shaping and the scored rule disagreed



10. The conflict rule is a **hard threshold** at ALT_MIN_SEP; our potential was **linear** in d_{alt} . One flight level — which leaves the pair squarely in conflict — collected **half the credit**.



11. A terminal state is worth **0**. If every term is a cost, crashing beats flying on wherever the remaining cost exceeds the charge. Ours was 6 against a median 9.4 — failures went 3/20 to **12/20** in 150k steps. Test the *tail*: p99 133, max 182.

💡 → **severity comes from predict_conflicts, the KPI's own quantity**

So the two definitions cannot drift apart. And the collision charge is $\text{collision_rate} \times$ **the steps not flown**, in units of w_{safe} : every episode completes to the same horizon, so dying is never the cheap way out *and* returns stay comparable across lengths.

The confound that outlived all eleven

⚠ Congestion is not comparable across policies

A policy that dies at step 10 accumulates ten steps of conflicts. One that flies the full 50 accumulates fifty. **Dying looks safe.**

On the board, the **brick** scores congestion **551** against the trained agent's **170** — a factor of 3.2, which reads like a contest. It flies **12.5** steps before it hits something. Per 100 steps actually flown: **44.0** against **3.46**. The real gap is **13×**.

The same arithmetic *clears* the rule-based row: 44.8 steps against 49.1, so its 90 is real. **The check is the point, not the verdict.**

💡 Which is why the rank key opens with **failed**

Rank on safety alone and **crashing out becomes a strategy**. Failing to fly the sector is not a way to fly it safely.

Seen live: a run with the best congestion on the board ranked **last**, on 4 failed episodes out of 20.

The habit

Never quote a per-step KPI without the failure count and the episode length beside it. Two of the three numbers are meaningless alone.

**Extra — shaping, curves, and
other things that lied**

Potential-based shaping — the only safe kind, and its trap

A shaping term of the form

$$F(s, s') = \gamma \phi(s') - \phi(s)$$

provably leaves the optimal policy unchanged (Ng, Harada & Russell, 1999).

It can make the problem easier to *learn* but cannot change what “solved” means. A hand-rolled “bonus for climbing” carries no such guarantee. Flight4DEnv does not use it — predicted severity is already dense — but Track C may.

⚠ The trap: both factors must be smooth

A potential that asks “*are these two within ALT_MIN_SEP?*” is a **step function**. ALT_MIN_SEP is **24**; one clearance moves an aircraft by DALT = **12**, so the first FL_INC leaves the pair inside the threshold and pays **exactly zero**. The agent is only paid on the second climb — the step it had no reason to reach. It survived three rounds of debugging because every piece was correct in isolation.

The test that would have caught it

Print the shaping reward for every single action from a typical state. If an action you *want* scores exactly zero, the potential is not smooth enough to teach it.

Check your agent is visiting the states your reward is about.

Shaping changes the value of states you visit. It can do nothing about states you never reach.

Two lies a training curve told us, on this exact environment

⚠ Lie 1 — “it is learning”

At 125k, ep_len_mean climbed 13.3 → 23.6. That looks exactly like progress.

Scored on the official KPI, the policy had collapsed to **7 clearances across 20 episodes**, with congestion *worse* than the brick.

It was climbing by learning to **stop acting**.

⚠ Lie 2 — “more steps will help”

The 800k unannealed run **unlearned** intervention monotonically: congestion 611 at 100k degrading to 770 at 800k, clearances 271 → 31.

The best checkpoint was the **earliest** one.

💡 Both were caught the same way

By scoring on the **official KPI** rather than trusting the training curve. That is one command, at any checkpoint: `python scripts/score_4d.py --agent <zip>`

Inference-time patches paper over training problems

A *refusal shield* was built to sit in front of the policy: a one-step look-ahead that vetoes a chosen clearance when it predicts a drop in reward or a critical conflict, substituting a safer action.

On an early, weak checkpoint it recovered a great deal of performance.

On the *properly trained* checkpoint, across 100 evaluation scenarios: **the raw policy was the strongest variant**. Every shield degraded it — one scenario's deviation blew out from 293 s to ≈ 1930 s — while intervening up to 80 times per episode.

⚠ The lesson

A post-hoc safety filter that *visibly rescues* your agent is evidence the agent is **not trained yet**.

It is a diagnostic, not a solution. Once the policy is genuinely good, the filter is overriding decisions that were better than its own.

💡 How to use it properly

Keep the shield as an **instrument**: measure how often it fires. A falling intervention rate is a clean, cheap training-progress signal that does not depend on your reward being right.

An entropy coefficient fails in both directions here

⚠ Too high

The clearance rate inflates until `max_clearances` truncates the episode before the horizon.

Measured once with the clearance cost annealed to zero: entropy $4.43 \rightarrow 9.34$ nats, episode length $50 \rightarrow 31.9$.

⚠ Too low

The policy collapses onto idling and never explores past it — and with `idle_bias`, the deterministic rollout is the brick.

💡 Control the quantity, not the coefficient

`--auto-entropy --auto-entropy-target 0.35` tunes `ent_coef` in log space so the policy's entropy tracks a target fraction of $\ln(n_{\text{actions}})$. Neither failure was about the coefficient; both were about the entropy.

The general rule

Entropy expands into any dimension the return does not push back on. Never zero a term you rely on to constrain the policy — anneal from a small floor if you want the curriculum at all.

Six things to take away

1. **Check the agent visits the states your reward is about.** 63% of conflicts happen after an untrained policy has already been sent home. No reward change reaches them.
2. **For every reward term, ask: rate or stock?** A cumulative total inside a per-step penalty charged nine times more for acting early than late.
3. **The reward and the score must agree.** Ours charged the detour; the ranking charged the endpoint. The scored quantity was 0.39% of what the reward penalised.
4. **Shaping must be potential-based *and* smooth.** A threshold potential paid exactly zero for the one action that started the solution.
5. **An exploration failure looks exactly like a reward failure.** 99.3% of first moves out of NOOP hurt. The optimum was always in the right place.
6. **Report the whole row.** The lowest-congestion agent on our board fails 96 episodes out of 100.

Handbook: sarl-plus.github.io/RL_bootcamp_2026 · every number here:
[slides/data/air_*.csv](#)